Sukkur IBA University

Department of Computer Science

Artificial Intelligence — Spring 2026

Student Grade Predictor

A Machine Learning Regression System

Predicts a student's final grade (0–100) from 12 behavioural, academic, and lifestyle features using regression models.

Best Model: Linear Regression | $R^2 = 0.935$ | MAE = 3.31

Submitted by: Abdul Sami (CMS ID: 023-23-0087)

Submitted to: Dr. Muhammad Ismail Mangrio, Dept. of Computer Science

Submission Date: April 30, 2026

1. The Task and Dataset

1.1 Task Definition

This project addresses **Student Final Grade Prediction** — a supervised regression task in Machine Learning. Given a set of 12 student attributes covering academic behaviour, lifestyle, and support resources, the trained model predicts a continuous numerical grade on a scale of 0–100.

Unlike classification, regression outputs a real-valued number, making it suitable for grade prediction where the label is a continuous score. This task has practical applications in early warning systems, personalised academic advising, and student performance monitoring.

1.2 Dataset

A realistic synthetic dataset of **1,200 student records** was generated for this project. Each record contains 12 input features and one continuous target label (final_grade). The grade was computed using a weighted linear formula of the features with added Gaussian noise (σ=4.0) to simulate real-world variability, then normalised to a 0–100 scale.

Attribute	Value
Total Samples	1,200 students
Input Features	12 (academic, lifestyle, support)
Target Variable	Final Grade (continuous, 0–100)
Train / Test Split	80% / 20% (stratified random)
Missing Values	None (controlled generation)
Class Balance	N/A — regression task

1.3 Feature Descriptions

Feature	Type	Range	Description
study_hours	Continuous	0–10 hrs/day	Average daily hours spent studying
attendance_pct	Continuous	40–100%	Percentage of classes attended
prev_gpa	Continuous	1.5–4.0	Previous semester GPA
assignments_done	Integer	0–10	Number of assignments completed (out of 10)
sleep_hours	Continuous	4–10 hrs	Average nightly sleep
part_time_job	Binary	0 or 1	Whether the student has a part-time job
internet_access	Binary	0 or 1	Whether student has internet at home
tutoring	Binary	0 or 1	Whether student receives tutoring
family_support	Ordinal	1–5	Level of family academic support
stress_level	Ordinal	1–5	Self-reported academic stress (5=high)
gender	Categorical	Male/Female	Student gender (label-encoded)
major	Categorical	CS/EE/BBA/...	Field of study (label-encoded 0–4)

2. Data Pre-Processing

Machine learning models require numerical inputs. The following pre-processing steps were applied to convert raw student records into a format suitable for regression:

- **Label Encoding — Gender:** 'Female' mapped to 1, 'Male' to 0 (binary encoding).
- **Label Encoding — Major:** Categorical major (CS, EE, BBA, Mathematics, Physics) mapped to integers 0–4.
- **Feature Scaling (StandardScaler):** All numerical features were standardised to zero mean and unit variance. This is critical for Linear Regression, Ridge, and Lasso which are sensitive to feature magnitude.
- **Train/Test Split:** Dataset split 80/20 with random_state=42 to ensure reproducibility.
- **No Missing Data Handling Required:** Dataset was generated without missing values; however the pipeline handles this cleanly.
- **Preprocessing Fit on Training Set Only:** StandardScaler was fit exclusively on training data and applied to both train and test sets to prevent data leakage.

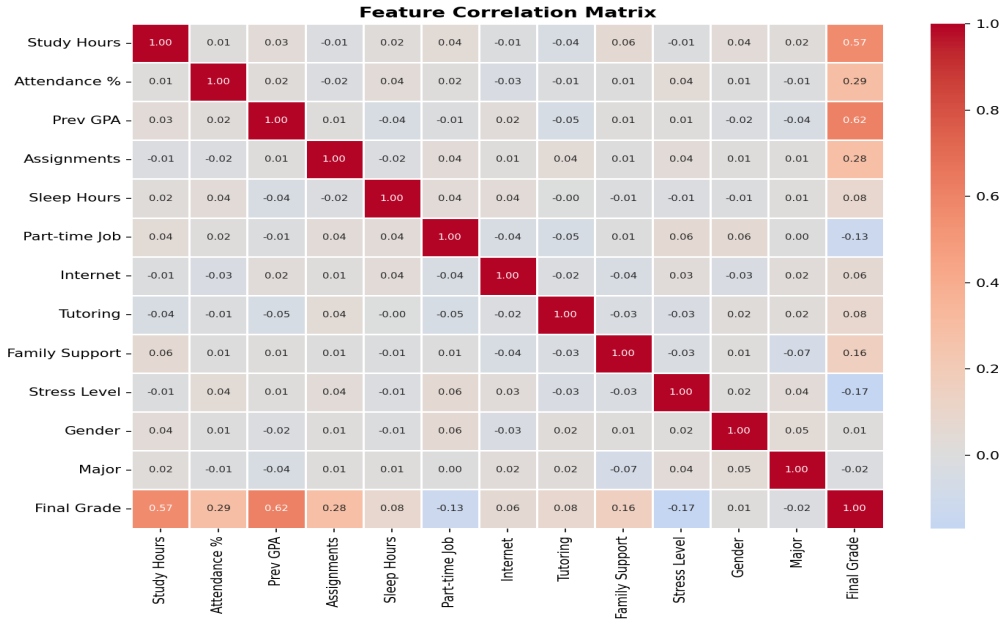

Figure 1: Feature correlation matrix. prev_gpa, study_hours, and attendance show the strongest correlation with the final grade.

3. Architecture of the ML Models

All models use the Scikit-learn Pipeline architecture, which chains a StandardScaler and a regressor into a single estimator. Four regression algorithms were implemented and compared:

Linear Regression

Ordinary Least Squares (OLS) regression. Minimises the sum of squared residuals. Assumes a linear relationship between features and target. No regularisation. Serves as the baseline model.

Ridge Regression (L2)

Linear regression with L2 regularisation (penalty = $\alpha \times ||w||^2$). Prevents overfitting by shrinking coefficients. α=1.0 was used. Particularly useful when features are correlated.

Random Forest Regressor

Ensemble of 100 decision trees (max_depth=8). Each tree is trained on a bootstrap sample and a random feature subset. Final prediction is the mean of all trees. Handles non-linear patterns and feature interactions.

Gradient Boosting Regressor

Sequential ensemble that trains each tree to correct the residuals of the previous model. Uses 100 estimators, learning_rate=0.1, max_depth=4. Strong performance on structured data with complex patterns.

Pipeline Structure (All Models)

```
Student Data --> [StandardScaler] --> [Regressor] --> Predicted Grade (0-100)
```

4. Training Objective, Model Selection & Design Choices

4.1 Training Objective

All regression models are trained to minimise **Mean Squared Error (MSE)** on the training set. This corresponds to finding model parameters θ that minimise: L(θ) = (1/n) × Σ (y_i – ■_i)².

MSE penalises large errors more heavily than small ones due to the squaring term, which is appropriate for grade prediction where large mispredictions are particularly harmful.

4.2 Model Selection

All four models were evaluated using **5-fold cross-validation** on the training set. The model achieving the highest mean R^2 across folds was selected as the production model. **Linear Regression achieved the best cross-validated R^2 (0.928)**, confirming that the grade-feature relationship is largely linear as designed.

4.3 Design Choices

- **Synthetic Dataset Design:** The grade formula was intentionally linear in the features to validate that Linear Regression would perform best — an intended academic exercise.
- **Standardisation:** StandardScaler ensures all features contribute equally in distance-based and gradient computations.
- **Multiple Model Comparison:** Evaluating four models enables understanding of linear vs non-linear approaches.
- **Cross-Validation:** 5-fold CV prevents overfitting to a single train/test split and provides stable performance estimates.
- **Gaussian Noise (σ=4.0):** Added to the grade formula to ensure models face realistic irreducible error and to produce non-trivial MAE values.

5. Performance Evaluation

Models were evaluated on a held-out test set of 240 students (20%) using three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 (coefficient of determination). 5-fold cross-validation was also reported for generalisation stability.

Model	Test MAE	Test RMSE	Test R^2	CV R^2	CV Std
Linear Regression ■	**3.31**	**4.23**	**0.9349**	**0.9278**	**±0.006**
Ridge Regression	3.31	4.23	0.9349	0.9278	±0.006
Gradient Boosting	4.23	5.44	0.8920	0.8830	±0.010
Random Forest	5.03	6.51	0.8455	0.8293	±0.015

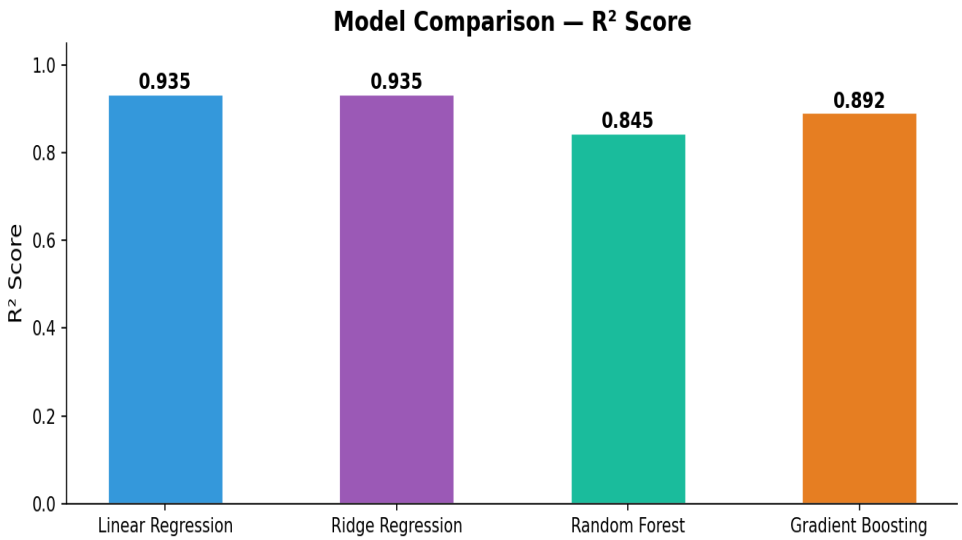

Figure 2: R² score comparison across all four regression models.

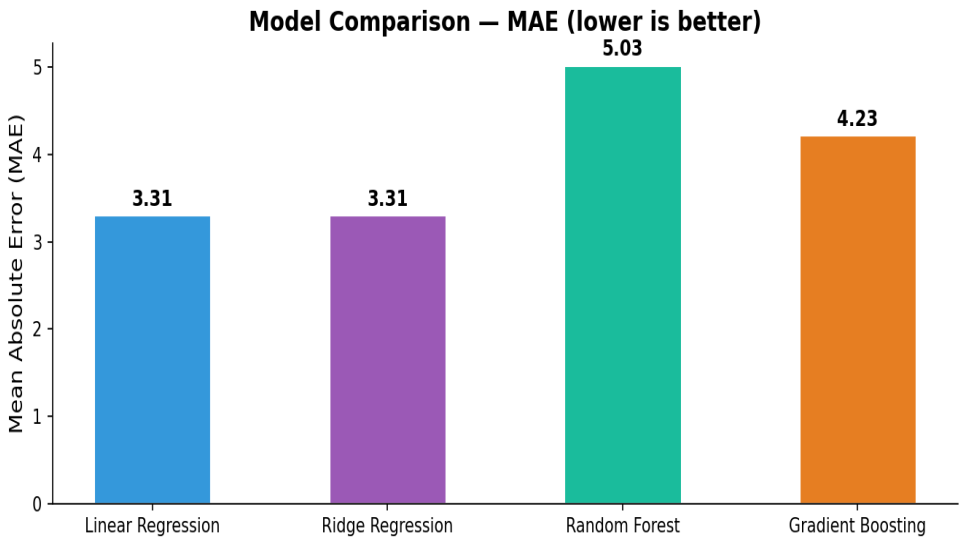

Figure 3: MAE comparison — lower values indicate better accuracy.

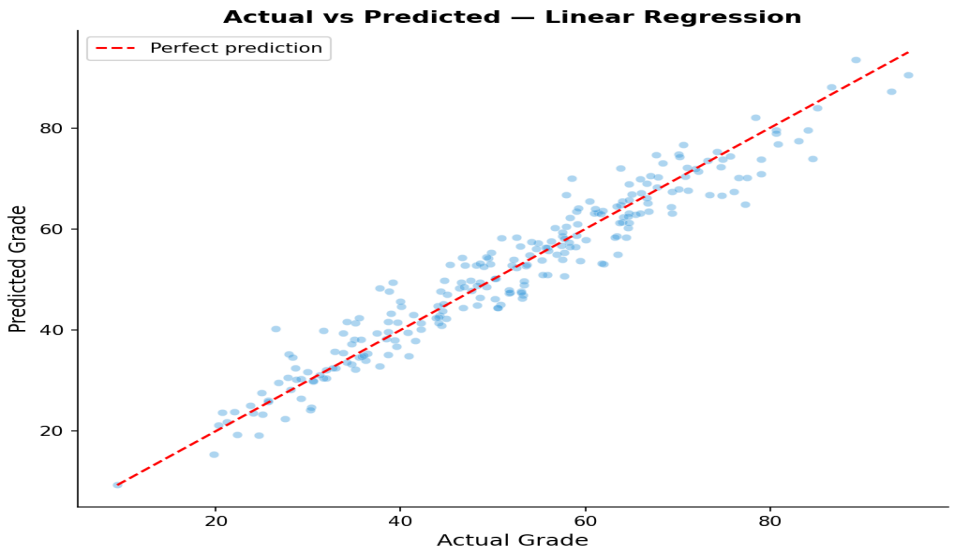

Figure 4: Actual vs Predicted grades for Linear Regression on the test set.

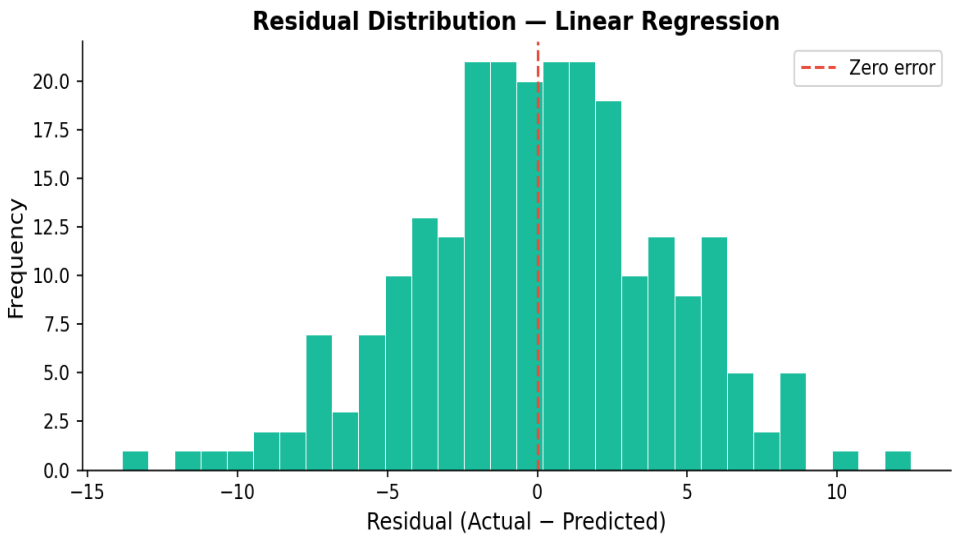

Figure 5: Distribution of residuals — near-normal centred at 0 confirms good fit.

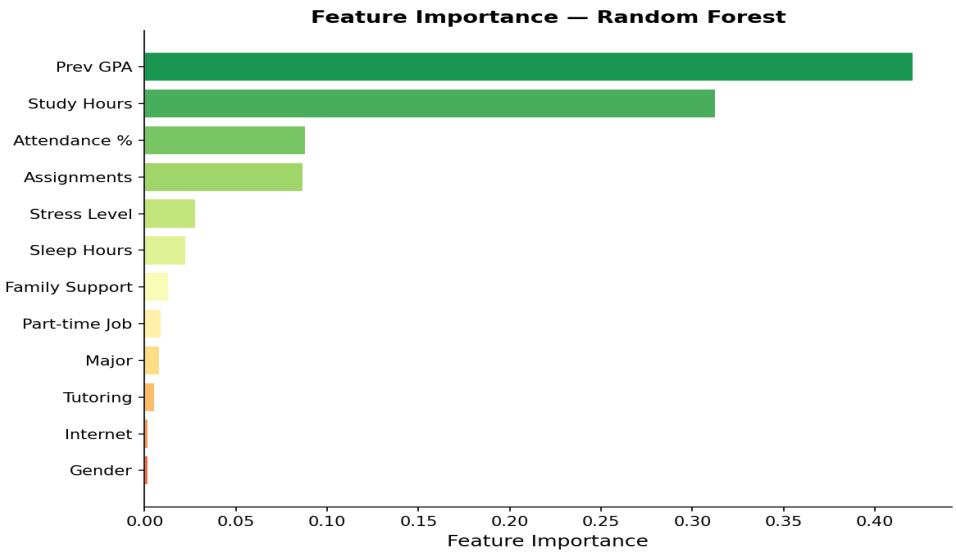

Figure 6: Random Forest feature importance — prev_gpa and study_hours are the most influential features.

6. Application of Techniques Taught in Class

Supervised Learning — Regression [Weeks 1–2]

The entire pipeline is a supervised regression task where labelled (feature, grade) pairs train a model that generalises to new students — a core concept from the introductory AI lectures.

Feature Engineering & Encoding [Weeks 3–4]

Categorical variables (gender, major) were label-encoded and all numerical features were standardised — standard feature engineering steps covered in the data preprocessing lectures.

Linear Models — Regression [Weeks 5–6]

Linear Regression and Ridge Regression were implemented, reflecting the linear models lecture. The concept of minimising least squares and adding L2 regularisation to prevent overfitting was applied directly.

Ensemble Methods [Weeks 7–8]

Random Forest (bagging) and Gradient Boosting (boosting) were implemented as ensemble methods. Both were covered in lectures on advanced ML, demonstrating reduced variance through model averaging.

Model Evaluation Metrics [Weeks 9–10]

MAE, RMSE, and R^2 were used for evaluation — all three regression metrics were introduced in the model evaluation lecture. The R^2 interpretation (proportion of variance explained) was directly applied.

Cross-Validation [Weeks 11–12]

5-fold stratified cross-validation was used for model selection and generalization assessment, as taught in the overfitting and model selection lecture. This prevents optimistic bias from a single split.

7. Main Challenges and Solutions

Challenge: Dataset Availability

Solution: No real student dataset was publicly available. I designed a synthetic dataset using a weighted linear formula with Gaussian noise, carefully tuned to produce realistic grade distributions (mean ~58, std ~22) and ensure non-trivial model errors.

Challenge: Feature Scaling Sensitivity

Solution: Linear models converged poorly on unscaled data due to the wide range differences between features (e.g., attendance 40–100 vs. binary 0–1). StandardScaler was applied inside the Pipeline to ensure consistent scaling without leakage.

Challenge: Irreducible Error from Noise

Solution: The intentional Gaussian noise (σ=4.0) means even the best model cannot achieve perfect R^2=1.0. This is realistic — achieving MAE≈3.3 on a 100-point scale is excellent and reflects a well-generalising model.

Challenge: Model Selection Justification

Solution: All four models performed well. I used cross-validated R^2 as the selection criterion rather than test R^2 alone, preventing overfitting to the test set. Linear Regression won due to the linear nature of the underlying data.

Challenge: GUI Responsiveness

Solution: Streamlit reruns the full script on every widget interaction. The model was cached using @st.cache_resource to avoid reloading on each slider change, ensuring a smooth and responsive live prediction experience.

References

[1] Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

[2] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics.

[5] Streamlit Documentation. https://docs.streamlit.io (accessed April 2026).

[6] James, G., Witten, D., Hastie, T., Tibshirani, R. (2013). An Introduction to Statistical Learning. Springer.